

Module 5: Software Assurance and Secure SQL

This is a working draft. It is not the submission PDF. Items marked pending require evidence from a live service or from GitHub.

Installation and packaging

The application supports Python 3.10 or newer. From `module_5/`, create a virtual environment with `python -m venv .venv`, then run `.venv/Scripts/python -m pip install -r requirements.txt` and `.venv/Scripts/python -m pip install -e .` on Windows. As an alternative, run `uv venv .venv`, `uv pip sync --python .venv/Scripts/python.exe requirements.txt`, and `uv pip install --python .venv/Scripts/python.exe -e .`. The `setup.py` metadata makes the local modules installable in editable mode, so imports work consistently in development, tests, and CI. Both install methods have been verified locally with successful `app`, `models`, and `query_data` imports.

SQL injection defenses and query limits

The analysis reads in `query_data.py` now have explicit `LIMIT` clauses. The SQLAlchemy analysis statements also use `.limit()` and bind their values through SQLAlchemy expressions. The new `lookup_applicants` function accepts only an approved search column, quotes the table and column with `psycopg.sql.Identifier`, and passes the search value through a placeholder and a separate parameter tuple. The SQL text never contains the user's value. The requested row count is clamped to the range 1 through 100, and the limit is passed as a bound parameter. A test with `Accepted ' OR 1=1 --` verifies that the malicious value stays in the parameter tuple and never enters the SQL statement. The loader also uses `sql.Identifier` when it needs to compose a database name.

Least privilege database role

The application now reads `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, and `DB_PASSWORD` from the environment, with `DATABASE_URL` and legacy `PG*` variables as alternatives. The example file `.env.example` has placeholders, and `.gitignore` excludes `.env`. The `gradcafe_app` role should receive only `CONNECT` on the database, `USAGE` on schema `public`, `SELECT` and `INSERT` on `public.applicants`, and `USAGE` on the primary key sequence. `SELECT` is needed for analysis; `INSERT` and sequence use support the Pull Data action. It should not own the database or receive `CREATE`, `ALTER`, `DROP`, `UPDATE`, `DELETE`, or superuser privileges. The grant script is `least_privilege.sql`. Read-only connections no longer try to provision a database. The data loader must be run separately with an administrator role when it creates the schema.

Pending: apply the grant script to the intended local or course database and capture a privilege listing to confirm the actual role configuration.

The core grant statements are `GRANT CONNECT ON DATABASE gradcafe TO gradcafe_app;`, `GRANT USAGE ON SCHEMA public TO gradcafe_app;`, `GRANT SELECT, INSERT ON TABLE public.applicants TO gradcafe_app;`, and `GRANT USAGE ON SEQUENCE public.applicants_p_id_seq TO gradcafe_app;`.

Dependency graph (seven sentences)

`dependency.svg` was generated from `src/app.py` with `pydeps` and `Graphviz`. The `app` module defines Flask routes and asks `orm_queries` for the analysis shown on the page. `orm_queries` uses `models` for the applicant mapping and SQLAlchemy session access. `models` depends on SQLAlchemy and the PostgreSQL dialect, which connect the Python model to PostgreSQL. `scrape_refresh` joins the scraper, data loader, and model layer for the optional refresh action. `analysis_format` turns database results into display strings. Flask and its template helpers render the page, while the external SQLAlchemy and Flask nodes account for most of the graph's complexity.

Verification and CI

The local Module 5 test command passed with 132 tests, three skips, and 100% coverage. Pylint on `src/` returned 10.00/10 using `python -m pylint src --fail-under=10`. The repository root workflow `.github/workflows/ci.yml` has separate Pylint, dependency graph, Snyk, and Pytest jobs. It runs on pushes and pull requests that change Module 5 or the workflow. The Snyk job requires the `SNYK_TOKEN` repository secret. A successful GitHub Actions run and screenshot are pending until the workflow is committed and pushed.

Snyk status

Snyk CLI 1.1307.4 successfully scanned `requirements.txt` after browser authentication and installation of the project dependencies. The scan completed with exit code 0 and tested 52 dependencies; it reported zero issues and no vulnerable paths. The original output is saved in `snyk-analysis.txt`, and the machine-readable result is `snyk-results.json`. The screenshot `snyk-analysis.png` shows the saved CLI output in a browser. This result describes known dependency issues at scan time, not a guarantee that the application has no security defects. No dependency remediation was required by this scan.